

Лабораторная работа №2

Коляды Екатерины z3342

24 апреля 2026 г.

Содержание

1	Цель работы	2
2	Задание	2
3	Реализация на C++	2
4	Реализация на Python	3
5	Сравнение быстродействия	3
6	Вывод	4

1 Цель работы

Познакомиться с принципами компиляции исходного кода. Составить программу с использованием циклов, условий и функций. Сравнить быстродействие между C++ и Python. Ознакомиться с типами данных.

2 Задание

Создать программы на C++ и Python, которые:

- Получают на вход количество итераций
- Выполняют вычисление: $x^2 - x^2 + x^4 - x^5 + x + x$
- Фиксируют время выполнения
- Имеют защиту от ввода нечисловых значений

3 Реализация на C++

```
C++ Labs > Lab2 > main.cpp > ...
1  #include <iostream>
2  #include <chrono>
3  #include <cmath>
4  using namespace std;
5  using namespace std::chrono;
6  //g++ main.cpp -o build/main.exe -fexec-charset=GBK -g
7  //.\build\main.exe
8
9  double calculate(double x) {
10     return pow(x, 2) - pow(x, 2) + pow(x, 4) - pow(x, 5) + x;
11 }
12
13 int main() {
14     int p;
15     double x = 1.5; // значение x для вычислений
16
17     while (true) {
18         cout << "Enter number of iterations: ";
19         cin >> p;
20         if (cin.fail()) {
21             cout << "Error: not a number! Program terminated." << endl;
22             break;
23         }
24
25         // Замер времени
26         auto start = high_resolution_clock::now();
27
28         // Циклический вызов функции calculate
29         for (int i = 0; i < p; i++) {
30             double result = calculate(x);
31             // Можно вывести результат для проверки (закомментировано, чтобы не спамить)
32             // cout << "Итерация " << i+1 << ": " << result << endl;
33         }
34
35         auto end = high_resolution_clock::now();
36         auto duration = duration_cast<microseconds>(end - start);
37
38         cout << "Time " << p << " iterations: " << duration.count() << " microseconds" << endl;
39         cout << "-----" << endl;
40     }
41
42     return 0;
43 }
```

Рис. 1:

4 Реализация на Python

```
1 import time
2
3 def calculate(x: float) -> float:
4     return x**2 - x**2 + x**4 - x**5 + x + x
5
6
7 def main():
8     x = 1.5
9
10    while True:
11        try:
12            p = int(input("\nNumber of iterations : "))
13
14        except ValueError:
15            # Если введено не число – завершаем программу
16            print("Error: not a number! ")
17            break
18
19        start_time = time.perf_counter()
20
21        for _ in range(p):
22            result = calculate(x)
23
24        end_time = time.perf_counter()
25
26        duration_us = (end_time - start_time) * 1_000_000
27
28        print(f" Time: {duration_us:.2f} microseconds")
29        print("-" * 50)
30
31
32 if __name__ == "__main__":
33     main()
```

Рис. 2:

5 Сравнение быстродействия

Язык	10 ⁶ итераций	10 ⁷ итераций
C++	~ 171100	~ 1643740 мкс
Python	~ 190105 мкс	~1854046 мкс

Таблица 1: Сравнение времени выполнения

6 Вывод

В ходе лабораторной работы:

- Изучены принципы компиляции C++ кода
- Реализованы программы с циклами и условиями
- Проведено сравнение быстродействия C++ и Python
- Настроена система контроля версий Git
- Созданы скрипты для деплоя